

06 — VACUUM

Module: PostgreSQL Internals | **Difficulty:** Advanced | **Est. reading time:** 55 min

Table of Contents

1. [Learning Objectives](#)
 2. [Why Dead Tuples Accumulate](#)
 3. [VACUUM: What It Does](#)
 4. [VACUUM vs VACUUM FULL](#)
 5. [Visibility Map and VACUUM](#)
 6. [Free Space Map Updates](#)
 7. [Freeze: Preventing XID Wraparound](#)
 8. [Vacuum Cost-Based Delay](#)
 9. [VACUUM Phases \(PG 9.6+\)](#)
 10. [ASCII Diagrams](#)
 11. [Live Observation Queries](#)
 12. [Common Mistakes](#)
 13. [Best Practices](#)
 14. [Performance Considerations](#)
 15. [Interview Questions](#)
 16. [Exercises with Solutions](#)
 17. [Cross-References](#)
-

Learning Objectives

After completing this section you will be able to:

- Explain why MVCC UPDATE creates dead tuples and how they accumulate.
 - Describe every action VACUUM takes on a heap page and in the FSM/VM.
 - Distinguish VACUUM from VACUUM FULL in terms of locking, I/O, and space reclamation.
 - Explain XID wraparound and how VACUUM freeze prevents it.
 - Configure cost-based delay to prevent VACUUM from starving production queries.
 - Use `pg_stat_user_tables` and `pg_stat_progress_vacuum` to monitor VACUUM.
-

Why Dead Tuples Accumulate

PostgreSQL implements **MVCC** (Multi-Version Concurrency Control) by keeping old row versions in the heap rather than overwriting them in place. This design allows readers to see a consistent snapshot without blocking writers.

What UPDATE actually does:

```
UPDATE orders SET status = 'shipped' WHERE id = 42;
```

1. A new tuple version is written with `t_xmin = current_xid`, `t_xmax = 0`.
2. The old tuple version has its `t_xmax` set to `current_xid` (marks it as deleted by this transaction).
3. The old tuple's `t_ctid` is updated to point to the new tuple location.
4. Both versions coexist in the same page (if space allows) or in different pages.

After the UPDATE commits:

- Transactions with a snapshot older than the UPDATE's XID still see the old version.
- Once NO active transaction can see the old version, it becomes a **dead tuple**.
- Dead tuples occupy space and waste I/O (they must be read during table scans).

DELETE sets `t_xmax` on the existing tuple with no new version created. The tuple becomes dead once the DELETE commits and no snapshot needs to see it.

VACUUM: What It Does

Regular VACUUM (non-FULL) is a multi-phase operation that reclaims dead tuples without rewriting the table:

Phase 1: Heap scan (identify dead tuples)

- Scans heap pages (skipping pages marked all-visible in the VM).
- For each tuple: checks `t_xmax` visibility. If the deleting transaction is committed and the XID is older than all active snapshots (`OldestXmin`), the tuple is dead.
- Builds a list of dead tuple TIDs to remove.

Phase 2: Index cleanup

- For each index on the table: removes index entries pointing to the dead TIDs.
- This is the most expensive part for tables with many indexes.

Phase 3: Heap cleanup

- Returns to each heap page that had dead tuples.
- Sets `lp_flags = LP_DEAD` on dead ItemIds.
- Compacts live tuples toward the end of the page (or uses `lp_flags = LP_REDIRECT` for HOT chains).
- Updates the **Free Space Map (FSM)** with the new available space.
- Sets the **all-visible bit** in the Visibility Map (VM) for pages where all remaining tuples are visible to all transactions.

Phase 4: Truncation (conditional)

- If trailing pages at the end of the heap file are entirely empty (all ItemIds unused or dead), VACUUM can physically truncate the file to release space back to the OS.
- This is the **only** way regular VACUUM returns space to the filesystem.
- VACUUM cannot remove empty pages in the middle of the file — only VACUUM FULL can do that.

VACUUM vs VACUUM FULL

Feature	VACUUM	VACUUM FULL
Lock required	SHARE UPDATE EXCLUSIVE (allows reads and writes)	ACCESS EXCLUSIVE (blocks all access)
Rewrites table	No	Yes — writes entirely new file
Reclaims mid-file space	No (marks as reusable internally)	Yes — packs all live tuples into minimal pages

Returns space to OS	Only trailing pages (truncation)	Yes — new file is exactly the right size
Index rebuild	Cleans index entries	Rebuilds all indexes
Duration	Short to medium	Potentially hours on large tables
WAL generated	Moderate	Very high (full table rewrite)
Use when	Routine maintenance	Table has severe bloat and maintenance window available

```
-- Regular vacuum (online, concurrent)
VACUUM orders;

-- Vacuum with analyze (updates statistics too)
VACUUM ANALYZE orders;

-- Full vacuum (offline, rewrites table – avoid on large tables in production)
VACUUM FULL orders;

-- Verbose output
VACUUM VERBOSE orders;

-- Freeze all tuples (aggressive anti-wraparound)
VACUUM FREEZE orders;
```

Visibility Map and VACUUM

The **Visibility Map (VM)** has two bits per heap page:

- **All-visible bit:** All tuples on this page are visible to all active transactions. VACUUM can skip reading this page during future cleanup passes (it has nothing to do there).
- **All-frozen bit:** All tuples on this page have been frozen (XID replaced with FrozenTransactionId). No future VACUUM freeze pass needs to visit this page.

VACUUM sets the all-visible bit for a page after confirming all tuples are visible. This dramatically speeds up future VACUUM runs — most of the heap can be skipped via the VM.

Free Space Map Updates

After removing dead tuples from a page, VACUUM updates the **Free Space Map (FSM)** to record how many bytes are now available in that page. The FSM is a compact tree structure that allows INSERT and HOT_UPDATE to quickly find a page with enough free space.

If VACUUM does not run, the FSM's free-space estimate becomes stale and INSERT must extend the table file rather than reusing pages, causing unnecessary bloat.

Freeze: Preventing XID Wraparound

The XID wraparound problem

Transaction IDs (XIDs) are 32-bit integers. With a 32-bit counter, the database can have at most ~4.2 billion transactions before the counter wraps around to zero. In PostgreSQL's MVCC model, wrapping around means very old transactions would suddenly appear to be in the future — causing data loss (old tuples would become invisible).

How freeze works

VACUUM can **freeze** tuples by replacing their `t_xmin` with `FrozenTransactionId` (2). A frozen XID is always treated as committed and always visible to all snapshots, so it never needs to be compared to the current XID. Once frozen, the tuple's visibility does not depend on the XID counter.

Normal tuple: `t_xmin = 16000042` → checked against snapshot `OldestXmin`
Frozen tuple: `t_xmin = 2` (`FrozenXID`) → always visible, no counter check needed

Freeze parameters

Parameter	Default	Effect
<code>vacuum_freeze_min_age</code>	50,000,000	Tuples older than 50M XIDs get frozen during regular VACUUM
<code>vacuum_freeze_table_age</code>	150,000,000	Force a full-table freeze scan when table's <code>relfrozenxid</code> is this many XIDs behind
<code>autovacuum_freeze_max_age</code>	200,000,000	Force autovacuum even if table is below normal thresholds — prevents wraparound

Emergency autovacuum

When a table's age (`current_xid - relfrozenxid`) exceeds `autovacuum_freeze_max_age` (200M transactions), PostgreSQL forces an aggressive autovacuum to freeze the table, even if autovacuum would normally skip it. This cannot be turned off.

```
-- Check how close tables are to wraparound (measure "age" in XIDs)
SELECT
  n.nspname      AS schema,
  c.relname      AS table,
  age(c.relfrozenxid)          AS xid_age,
  2000000000 - age(c.relfrozenxid) AS xids_until_emergency_vacuum,
  c.relfrozenxid
FROM pg_class c
JOIN pg_namespace n ON n.oid = c.relnamespace
WHERE c.relkind = 'r'
ORDER BY age(c.relfrozenxid) DESC
LIMIT 20;
```

Vacuum Cost-Based Delay

VACUUM reads and writes a large number of pages, which can saturate I/O and increase query latencies. Cost-based delay throttles VACUUM by making it sleep after it has "spent" a certain number of I/O cost units.

Parameter	Default	Effect
<code>vacuum_cost_delay</code>	0 ms	0 = no delay (vacuum runs at full speed). Set to 2–20 ms for throttling.
<code>vacuum_cost_limit</code>	200	Cost units accumulated before sleeping
<code>vacuum_cost_page_hit</code>	1	Cost to read a page from shared_buffers
<code>vacuum_cost_page_miss</code>	10	Cost to read a page from disk (cache miss)
<code>vacuum_cost_page_dirty</code>	20	Cost to write a page (make it dirty)

For autovacuum specifically:

- `autovacuum_vacuum_cost_delay` (default 2 ms in PG 13+)
- `autovacuum_vacuum_cost_limit` (default -1, which means use `vacuum_cost_limit`)

```
-- Show cost delay settings
SHOW vacuum_cost_delay;
SHOW vacuum_cost_limit;
SHOW autovacuum_vacuum_cost_delay;
```

VACUUM Phases

PostgreSQL 9.6+ reports VACUUM progress via `pg_stat_progress_vacuum` :

Phase	Description
initializing	Starting up, reading relation
scanning heap	Scanning heap for dead tuples
vacuuming indexes	Removing dead entries from indexes
vacuuming heap	Removing dead tuples from heap pages
cleaning up indexes	Index bulk-delete cleanup
truncating heap	Removing trailing empty pages
performing final cleanup	Updating FSM, VM, stats

```
-- Watch live vacuum progress
SELECT
    p.pid,
    p.datname,
    p.relid::regclass AS table,
    p.phase,
    p.heap_blks_total,
```

```

p.heap_blks_scanned,
p.heap_blks_vacuumed,
p.index_vacuum_count,
p.num_dead_tuples,
p.max_dead_tuples
FROM pg_stat_progress_vacuum p;

```

ASCII Diagrams

Dead Tuple Accumulation and VACUUM

BEFORE VACUUM:

Page 0

ItemId[1] → LIVE	tuple (id=1, status='shipped')		
ItemId[2] → DEAD	tuple (id=1, status='pending')	←	← from UPDATE
ItemId[3] → LIVE	tuple (id=2, status='pending')		
ItemId[4] → DEAD	tuple (id=3, ...)	←	← from DELETE
pd_upper = 7200 (lots of wasted space in dead tuples)			
pd_lower = 40 (4 ItemIds × 4 bytes + 24 header)			

AFTER VACUUM:

Page 0

ItemId[1] → LIVE	tuple (id=1, status='shipped')
ItemId[2] → UNUSED	(LP_DEAD → can be reused)
ItemId[3] → LIVE	tuple (id=2, status='pending')
ItemId[4] → UNUSED	(LP_DEAD → can be reused)
pd_upper = 7520 (free space increased!)	
FSM updated: this page has ~300 bytes free	
VM bit set if all remaining tuples are visible	

XID Wraparound Prevention

XID counter (32-bit, modular arithmetic):

```

0 ----- 4,294,967,295
|
|   FrozenXID=2               current_xid
|   ▼                       ▼
|   2 ----- 16000000 ----- 200000000 ----- 2000000000 → wrap back to 0?
|
| VACUUM FREEZE replaces t_xmin with 2 (FrozenXID)
| for tuples older than vacuum_freeze_min_age (50M XIDs)
|

```

| Autovacuum fires when table age > autovacuum_freeze_max_age (200M XIDs)
| to prevent counter ever reaching 2^{31} away from current XID

Live Observation Queries

```
-- 1. Dead tuple stats per table
SELECT
    schemaname,
    relname,
    n_live_tup,
    n_dead_tup,
    CASE WHEN n_live_tup + n_dead_tup > 0
        THEN round(100.0 * n_dead_tup / (n_live_tup + n_dead_tup), 1)
        ELSE 0 END AS dead_pct,
    last_vacuum,
    last_autovacuum,
    vacuum_count,
    autovacuum_count
FROM pg_stat_user_tables
ORDER BY n_dead_tup DESC
LIMIT 20;

-- 2. Table bloat estimate (requires pgstattuple extension)
CREATE EXTENSION IF NOT EXISTS pgstattuple;
SELECT * FROM pgstattuple('orders');
/*
table_len | tuple_count | tuple_len | tuple_percent | dead_tuple_count
          | dead_tuple_len | dead_tuple_percent | free_space | free_percent
*/

-- 3. XID age / wraparound risk
SELECT
    datname,
    age(datfrozenxid) AS database_age,
    2000000000 - age(datfrozenxid) AS xids_until_emergency
FROM pg_database
ORDER BY age(datfrozenxid) DESC;

SELECT
    schemaname, relname,
    age(relfrozenxid) AS table_age,
    relfrozenxid
FROM pg_class c
JOIN pg_namespace n ON n.oid = c.relnamespace
WHERE relkind = 'r'
ORDER BY age(relfrozenxid) DESC
LIMIT 10;

-- 4. Watch live VACUUM progress
SELECT pid, datname, relid::regclass, phase,
```

```

        heap_blks_total, heap_blks_scanned, heap_blks_vacuumed,
        num_dead_tuples
FROM pg_stat_progress_vacuum;

-- 5. Manually trigger vacuum on a specific table
VACUUM VERBOSE ANALYZE orders;
-- Sample output:
-- INFO:  vacuuming "public.orders"
-- INFO:  scanned index "orders_pkey" to remove 1523 row versions
-- INFO:  "orders": removed 1523 row versions in 89 pages
-- INFO:  "orders": found 1523 removable, 45821 nonremovable row versions in 512
pages

-- 6. Check cost delay settings
SELECT name, setting, unit
FROM pg_settings
WHERE name LIKE '%vacuum%cost%' OR name LIKE '%autovacuum%cost%'
ORDER BY name;

-- 7. Detect tables needing urgent vacuum (high dead tuple ratio)
SELECT relname,
       n_dead_tup,
       n_live_tup,
       round(100.0 * n_dead_tup / NULLIF(n_live_tup + n_dead_tup, 0), 2) AS
bloat_pct,
       pg_size_pretty(pg_total_relation_size(relid)) AS total_size
FROM pg_stat_user_tables
WHERE n_dead_tup > 1000
      AND round(100.0 * n_dead_tup / NULLIF(n_live_tup + n_dead_tup, 0), 2) > 10
ORDER BY bloat_pct DESC;

```

Common Mistakes

1. **Disabling autovacuum.** Never set `autovacuum = off` unless you are 100% certain you have a manual maintenance strategy that prevents wraparound. Many production outages have been caused by this.
 2. **Forgetting TOAST tables need vacuuming.** Dead chunks in TOAST tables accumulate just like dead tuples in main tables. Regular VACUUM on the main table also processes the associated TOAST table.
 3. **Using VACUUM FULL during production hours.** VACUUM FULL holds ACCESS EXCLUSIVE lock, blocking all reads and writes. Schedule it only during maintenance windows.
 4. **Confusing TRUNCATE with VACUUM.** TRUNCATE instantly removes all rows and resets the table to empty. It is not related to dead tuple cleanup. VACUUM handles incremental dead tuple removal.
 5. **Ignoring n_mod_since_analyze in pg_stat_user_tables.** A table can be vacuumed but not analyzed — stale statistics will lead to bad query plans.
 6. **Relying on pg_relation_size to detect bloat.** Regular VACUUM does not shrink the file (except trailing truncation). Use `pgstattuple` or check `n_dead_tup` for actual bloat measurement.
-

Best Practices

- **Keep autovacuum enabled** and monitor `pg_stat_user_tables` for tables not being autovacuumed.
- **Run `VACUUM ANALYZE`** rather than just `VACUUM` to keep statistics fresh.
- **Use `vacuum_cost_delay = 2-10 ms`** in `autovacuum_vacuum_cost_delay` to throttle autovacuum without starving it.
- **Tune per-table autovacuum** for high-write tables using storage parameters:

```
ALTER TABLE hot_table SET (autovacuum_vacuum_scale_factor = 0.01);
```

- **Monitor wraparound risk** for all databases including non-production ones.
- **Set `autovacuum_freeze_max_age = 1500000000`** (1.5 billion) on busy OLTP systems to give more headroom.

Performance Considerations

Scenario	Impact	Solution
High UPDATE rate	Dead tuple accumulation → bloat → slow scans	Lower <code>autovacuum_vacuum_scale_factor</code> for that table
autovacuum can't keep up	<code>n_dead_tup</code> grows unbounded	Increase <code>autovacuum_vacuum_cost_limit</code> , decrease <code>autovacuum_vacuum_cost_delay</code>
Expensive index vacuum	Many indexes slow down VACUUM	Limit indexes to those actually used
VACUUM FULL contention	ACCESS EXCLUSIVE blocks all queries	Use <code>pg_repack</code> extension for online table rewriting
XID wraparound risk	Emergency autovacuum + forced freezes	Run periodic <code>VACUUM FREEZE</code> on old tables
VACUUM generating too much WAL	Impacts replication lag	Unavoidable; but schedule large VACUUMs during low-traffic periods

Interview Questions

Q1. Why does PostgreSQL UPDATE create a dead tuple?

Because PostgreSQL uses MVCC. UPDATE does not overwrite the existing row; instead, it writes a new tuple version and marks the old version as deleted (sets `t_xmax`). The old version must remain until no active transaction snapshot can see it, at which point it becomes a dead tuple. VACUUM eventually reclaims this space.

Q2. What exactly does VACUUM do to a heap page?

For pages with dead tuples: removes index entries pointing to dead TIDs, marks dead `ItemId` slots as `LP_DEAD` / `LP_UNUSED`, updates the FSM with new free space, and optionally sets the all-visible bit in the VM. It does NOT physically compact the page or release the space to the OS (except for trailing pages).

Q3. What is XID wraparound and how does VACUUM prevent it?

Transaction IDs are 32-bit counters. After ~4 billion transactions, the counter would wrap around, causing old tuples to appear to be in the future and become invisible. VACUUM freeze prevents this by replacing `t_xmin` with `FrozenTransactionId` (2), a special value always treated as committed and always visible, removing the dependency on the counter's value.

Q4. What is the difference between `VACUUM` and `VACUUM FULL` ?

`VACUUM` requires only `SHARE UPDATE EXCLUSIVE` lock (reads and writes proceed), marks dead tuples as reusable within the same file, cannot compact mid-file gaps, and is fast. `VACUUM FULL` requires `ACCESS EXCLUSIVE` lock (blocks all access), rewrites the entire table into a new file, reclaims all space, and rebuilds all indexes — but takes much longer.

Q5. What does `vacuum_freeze_min_age` control?

It sets the minimum age (in transactions) of a tuple before VACUUM will freeze it by replacing `t_xmin` with `FrozenTransactionId`. Default is 50,000,000 XIDs. Tuples younger than this are left with their original `t_xmin` even during a freeze scan.

Q6. How does cost-based vacuum delay work?

VACUUM tracks I/O cost units: reading from buffer cache costs `vacuum_cost_page_hit` (1), reading from disk costs `vacuum_cost_page_miss` (10), writing dirty costs `vacuum_cost_page_dirty` (20). When accumulated cost reaches `vacuum_cost_limit` (200), VACUUM sleeps for `vacuum_cost_delay` milliseconds, then resets the counter. This prevents VACUUM from monopolising I/O.

Q7. What does `pg_stat_user_tables.n_dead_tup` represent?

The estimated number of dead (obsolete) tuples in the table. This is the primary metric for deciding when to run VACUUM and for detecting autovacuum health.

Q8. Why doesn't regular VACUUM return space to the OS?

Regular VACUUM marks dead tuple slots as reusable within the existing file but does not move live tuples to compact the file. Only trailing empty pages can be truncated (freed to OS). VACUUM FULL rewrites the entire table into a new, compact file and is the only reliable way to return mid-file space to the OS.

Exercises with Solutions

Exercise 1 — Observe dead tuple creation

```
-- Solution
CREATE TABLE vacuum_demo (id int, val text);
INSERT INTO vacuum_demo SELECT i, 'original_' || i FROM generate_series(1, 1000) i;

-- Baseline dead tuples
SELECT n_live_tup, n_dead_tup FROM pg_stat_user_tables WHERE relname =
'vacuum_demo';

-- Run 500 updates
UPDATE vacuum_demo SET val = 'updated_' || id WHERE id <= 500;

-- Check again (may need ANALYZE or wait for stats collector)
ANALYZE vacuum_demo;
```

```

SELECT n_live_tup, n_dead_tup FROM pg_stat_user_tables WHERE relname =
'vacuum_demo';
-- n_dead_tup should be ~500

-- Vacuum and confirm cleanup
VACUUM vacuum_demo;
ANALYZE vacuum_demo;
SELECT n_live_tup, n_dead_tup FROM pg_stat_user_tables WHERE relname =
'vacuum_demo';
-- n_dead_tup should be ~0

```

Exercise 2 — Measure table size before and after VACUUM FULL

```

-- Solution
CREATE TABLE bloat_example AS
SELECT i, md5(i::text) AS data FROM generate_series(1, 100000) i;
DELETE FROM bloat_example WHERE i % 2 = 0; -- delete half

-- Size with bloat
SELECT pg_size_pretty(pg_relation_size('bloat_example')) AS size_with_bloat;

-- Regular vacuum (won't shrink mid-file)
VACUUM bloat_example;
SELECT pg_size_pretty(pg_relation_size('bloat_example')) AS after_regular_vacuum;

-- Full vacuum (rewrites and compacts)
VACUUM FULL bloat_example;
SELECT pg_size_pretty(pg_relation_size('bloat_example')) AS after_full_vacuum;
-- Expected: ~50% of size_with_bloat

```

Exercise 3 — Check wraparound risk

```

-- Solution: find the oldest unfrozen tables
SELECT
    n.nspname      AS schema,
    c.relname      AS table,
    age(c.relfrozensid) AS xid_age,
    c.relfrozensid,
    pg_size_pretty(pg_total_relation_size(c.oid)) AS size,
    CASE
        WHEN age(c.relfrozensid) > 1800000000 THEN 'CRITICAL'
        WHEN age(c.relfrozensid) > 1500000000 THEN 'WARNING'
        WHEN age(c.relfrozensid) > 1000000000 THEN 'CAUTION'
        ELSE 'OK'
    END AS wraparound_risk
FROM pg_class c
JOIN pg_namespace n ON n.oid = c.relnamespace
WHERE c.relkind IN ('r', 't') -- tables and TOAST tables

```

```
ORDER BY age(c.relFrozenxid) DESC  
LIMIT 20;
```

Cross-References

- [02_heap_storage.md](#) — Dead tuple structure in heap pages
- [07_autovacuum.md](#) — Autovacuum tuning and monitoring
- [08_visibility_map.md](#) — VM all-visible bit set by VACUUM
- [09_free_space_map.md](#) — FSM updated by VACUUM to record free space
- [03_toast.md](#) — VACUUM processes TOAST tables alongside main tables